SCALER School of Technology | Priyansh Saxena

Feature Stores, Data Pipelines & Offline/Online Serving

Data Sources ▪ Formats ▪ Feature Engineering ▪ Feature Stores

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 2 — Session 1/2

ROADMAP

Agenda

1. Data Sources and Formats
2. Data Models and Storage
3. Batch vs. Stream Processing
4. Feature Engineering
5. Feature Stores

TOC

1

Data Sources and Formats

1 Where Does ML Data Come From?

User input data

- Text, images, videos, uploaded files
- If users *can* input wrong data, they *will*
- Requires heavy validation at ingest
- Users expect immediate results ⇒ fast processing needed

System-generated data

- Logs: system state, events, model predictions
- **Log everything** — "interesting" in production usually means a crash or an astronomical cloud bill
- Immutable audit trail for debugging

Internal databases

- Company assets, CRM records, inventory
- Typically structured, well-maintained
- Access via SQL or internal APIs

Third-party data

- **First-party**: your own users
- **Second-party**: another company's customers
- **Third-party**: aggregated from public sources
- Apple IDFA change (2021): made opt-in ⇒ devastated the $80B ad industry overnight

1 Data Formats: JSON, CSV, and Parquet

	JSON	CSV	Parquet
Storage layout	Key-value text	Row-major text	Column-major binary
Readability	Human-readable	Human-readable	Binary (not readable)
File size	Large (verbose)	Medium	Small (6× less than CSV on S3)
Best for	APIs, configs	Writing rows	Reading features
Speed	Slow to parse	Slow column reads	2× faster to unload vs. CSV
Tools	Ubiquitous	pandas, Excel	Spark, pandas, DuckDB

Why Parquet dominates ML pipelines

6× less storage and **2× faster reads** than CSV on S3. Column-major layout means you can read only the columns you need without scanning entire rows.

1 Row-Major vs. Column-Major: Why It Matters

Row-major (CSV, JSON)

- Rows stored contiguously on disk
- Fast to **write** one sample at a time
- Fast to **retrieve all features of one sample**
- Slow to read a single feature across all samples
- Good for: transactional writes, streaming inserts

Column-major (Parquet, ORC)

- Columns stored contiguously on disk
- Fast to **read one feature across all samples**
- Excellent compression (same-type data clusters)
- Good for: analytics, ML feature reads

The pandas consequence

pandas is built around column-major format internally.

```
Iterating a DataFrame by row:  2.41 s
Iterating a DataFrame by column:  0.07 s
```

34× slower to iterate by row. Always prefer vectorised column operations.

Rule of thumb

Writes ⇒ **row-major** (CSV, databases)
Feature reads ⇒ **column-major** (Parquet)

1 Structured vs. Unstructured Data

Structured data

- Well-defined schema: columns, types, constraints
- Lives in relational databases or data warehouses
- Easy to query with SQL
- Examples: user age, order amount, transaction count
- Typically **10–20%** of enterprise data

Unstructured data

- No predefined schema
- Text, images, audio, video, sensor readings
- Requires ML to extract signal
- Typically **80–90%** of all industry data
- Stored in object stores (S3, GCS) or data lakes

Industry reality

"In industry, 80–90% of data is unstructured." — Chip Huyen, *Designing Machine Learning Systems*
Most of an ML engineer's time is spent making unstructured data usable for models.

1 Quiz 1

Q1 — Format Selection

You are building a feature pipeline. You need to:

(a) Stream 10,000 user click events per second from your web app to a data lake

(b) Run nightly batch queries to compute the average session duration across 100M user rows

(c) Serve a REST API that returns a single user's profile as a response payload

Which storage format would you prefer for each use case, and why?

Q2 — Unstructured Data

A product team wants to build a sentiment classifier on customer support chat transcripts. The data lives in S3 as raw `.txt` files.

Name two data quality problems you would check for before training, and explain why each one matters.

1 Quiz 1 — Answers

A1 — Format Selection

(a) Streaming events → row-major / JSON: each event is a single row written immediately; low-latency ingestion matters more than read speed.

(b) Batch aggregate queries → Parquet (column-major): scanning one column (session duration) across 100M rows is 6× faster and cheaper than CSV.

(c) REST API response → JSON: human-readable, schema-flexible, universally supported by API clients.

A2 — Unstructured Data Quality

Problem 1 — Language/encoding issues: raw `.txt` chat files may mix encodings (UTF-8, Latin-1) or languages. A sentiment model trained on English will silently fail on non-English transcripts. Fix: detect and filter by language; normalise encoding at ingest.

Problem 2 — Label imbalance / missing labels: support chats are not uniformly distributed across sentiments. If 90% are negative, a model predicting "negative" always achieves 90% accuracy. Fix: audit label distribution; oversample or reweight minority classes.

2

Data Models and Storage



2 The Relational Model: Tables and Normalization

Core concepts

- Data organised into **relations** (tables) of tuples (rows)
- Each tuple has a unique key; relations linked via foreign keys
- **Normalization** reduces redundancy: 1NF, 2NF, 3NF
- Query via SQL — declarative, set-based

The Harry Potter publisher example

- Without normalization: publisher name stored in every book row
- Publisher changes name ⇒ update 4 rows — easy to miss one
- With normalization: one `Publishers` table; books reference by ID
- Update one row, consistent everywhere

When to use relational databases

- Structured data with clear schema
- Transactions requiring ACID guarantees
- Complex joins across entities
- E.g.: user accounts, orders, inventory

ML relevance

Most production ML systems have a relational backbone for structured metadata: user profiles, item catalogs, label stores. Features are derived from these tables.

2 NoSQL: Four Models for Different Problems

Type	Structure	Tools	Use in ML
Document	JSON-like documents; flexible schema	MongoDB, CouchDB	User profiles, content metadata
Key-value	Hash map; ultra-low latency	Redis, DynamoDB	Online feature store; session cache
Graph	Nodes and edges; relationship queries	Neo4j, Neptune	Fraud graph features; social network
Column-family	Wide rows; sparse columns	Cassandra, HBase	Time-series features; event streams

The online feature store pattern

The most common NoSQL use in ML: **Redis** as an online feature store. Pre-computed features are written to Redis at training time and read at sub-millisecond latency at serving time.

2 OLTP vs. OLAP: Two Different Access Patterns

OLTP — Online Transaction Processing

- **Optimised for writes** and point lookups
- Low latency, high concurrency
- Small transactions: one user, one order
- Examples: user signs up, order placed, payment processed
- Row-oriented storage
- Tools: PostgreSQL, MySQL, Aurora

OLAP — Online Analytical Processing

- **Optimised for reads** and aggregate queries
- Large scans across millions of rows
- "How many orders in the last month?"
- Examples: business reporting, feature aggregation, model evaluation
- Column-oriented storage
- Tools: Redshift, BigQuery, Snowflake, ClickHouse

ML data engineering in practice

Raw events land in OLTP. Nightly ETL moves data to OLAP. Feature engineering pipelines run against OLAP. Training data is materialized to Parquet in S3.

2 Moving Data Between Services

Via databases

- Service A writes to a shared DB
- Service B reads from it
- Simple, auditable
- Adds latency; poor for real-time
- Tight coupling on schema

Via services (REST/gRPC)

- A sends HTTP request to B's API
- RESTful APIs: JSON over HTTP
- gRPC: Protocol Buffers, binary, faster
- Synchronous: caller waits
- Good for request-response patterns

Via real-time transport (Kafka)

- Event-based, pub/sub model
- Producers write to topics; consumers read
- Data broker **decouples** producers from consumers
- Enables stream processing (Flink, Spark Streaming)
- Best for high-throughput, event-driven ML



5-Minute Break

You just stored data in three formats, four databases, and two processing paradigms.

Grab some water. Your feature pipeline needs you well-hydrated.

3

Batch vs. Stream Processing

3 Batch Processing vs. Stream Processing

Batch processing

- Periodically process large accumulated data
- High throughput; high latency (minutes to hours)
- Tools: MapReduce, Apache Spark
- When your data is processed in batch, it is **historical data**
- Examples: nightly training data aggregation, weekly model retrain
- Cheaper; simpler to operate

Stream processing

- Process data as each event arrives
- Low latency (milliseconds to seconds); lower throughput
- Tools: Apache Flink, Spark Streaming, Kafka Streams
- When processed in real-time, it is **streaming data**
- Examples: real-time fraud detection, live recommendation refresh
- More complex; stateful aggregations are hard

The key question: how fresh does the feature need to be?

Fraud detection: features must reflect the last few seconds of activity $\Rightarrow$ streaming.
Weekly demand forecast: historical aggregates are fine $\Rightarrow$ batch.

4

Feature Engineering

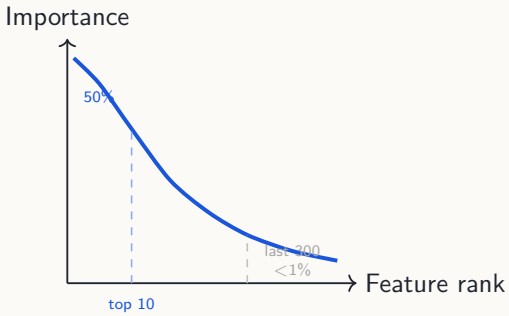
4 Why Features Matter More Than Algorithms

The Facebook finding (2014)

- Paper on ad click prediction at Facebook scale
- Key result: **top 10 features** accounted for ≈50% of model's total feature importance
- The **last 300 features** contributed less than 1%
- Implication: feature selection matters as much as model choice
- "Having the right features is the most important thing in developing their ML models"

Common operations

- Handling missing values
- Scaling and normalization
- Encoding categoricals
- Feature crossing



Adapted from Facebook 2014 ad CTR paper

4 Handling Missing Values and Scaling

Missing value strategies

- **Deletion**: drop rows or columns with nulls
  - Risk: introduces **selection bias** if data is not MCAR
- **Imputation**: fill with mean, median, mode, or model-predicted value
  - Risk: **injects noise** if imputation is wrong
  - Always impute on training stats only
- Neither is universally correct — understand *why* values are missing

Scaling methods

- **Normalization** $[0, 1]$: $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$
  Use when: bounded range, no outliers
- **Standardization** $\mu = 0, \sigma = 1$: $x' = \frac{x - \mu}{\sigma}$
  Use when: Gaussian-ish, outliers present

Critical: fit on training data only

Compute mean/std/min/max on the **training split**.
Apply those stats to val and test.
Fitting on the full dataset is data leakage.

4 Feature Crossing and Data Leakage

Feature crossing

- Combine two or more features to capture interactions
- Example: *marital status* $\times$ *number of children*
  - Better predictor of house-buying intent than either alone
- Common in logistic regression and tree ensembles
- Deep learning learns crosses implicitly; explicit crosses help shallow models
- Risk: exponential feature explosion if unconstrained

Data leakage: the silent accuracy inflation trap

Your validation accuracy is suspiciously high. Did you:

- Scale *before* splitting train/test?
- Use future information in a feature?
- Include the label in a proxy feature?

Rule: always split first, *then* compute feature statistics.

Quote

"It's hard avoiding data leakage without understanding how your data is collected." — Chip Huyen

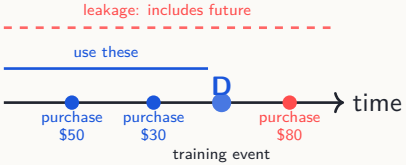
4 Point-in-Time Correctness

The problem

- You are building a training row for a recommendation event on **date D**
- Feature: "user's total purchases in the past 30 days"
- Wrong: compute this feature using *all available data up to today*
- Correct: compute using *only data available before date D*

Why it matters

- Violation = data leakage using future information
- Model appears to work perfectly in offline eval
- Fails completely in production where future is unavailable
- Feature store enforces this automatically



Feature stores perform **point-in-time joins**: for each training row at time t, return only features computed from data $< t$.

5

Feature Stores

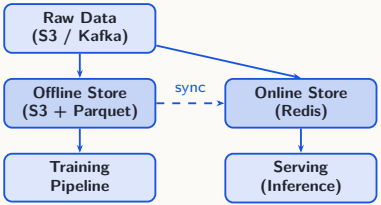
5 The Feature Store: Offline + Online Architecture

Offline store (training)

- Historical feature values for model training
- Storage: S3 + Parquet / Delta Lake
- Point-in-time correct feature retrieval
- Prevents training-serving skew
- Batch compute: Spark, dbt

Online store (serving)

- Pre-computed features for real-time inference
- Storage: Redis, DynamoDB (key-value)
- Sub-millisecond reads
- Must stay in sync with offline store
- Staleness budget: how stale is acceptable?



Compute once, use everywhere

Feature logic defined *once*. Training and serving use identical computations — no training-serving skew.

5 Why Separate Offline and Online?

The problem without separation

- Training: read features from S3 batch job (cheap, slow)
- Serving: recompute features from database on every request (expensive, fragile)
- Result: different feature values at train vs. serve time
- Debugging: impossible to reproduce a production prediction

The problem with only a database

- Production DB cannot sustain model training reads (millions of rows)
- Serving queries would be slow (complex aggregations)
- No historical versioning — what were the features on Jan 1st?

What separation gives you

1. **Training**: point-in-time correct historical features
2. **Serving**: sub-millisecond reads for online inference
3. **Consistency**: same feature definitions, same values
4. **Reuse**: team A's feature is reusable by team B
5. **Auditability**: reproduce any prediction from any date

Tools: Feast (open source), Tecton (managed), Hopsworks (managed/open)

5 Feature Store Tools and Demo Teaser

Tool	Type	Offline store	Online store
Feast	Open source	S3, GCS, Redshift, BigQuery	Redis, DynamoDB, SQLite
Tecton	Fully managed (SaaS)	S3 + Delta Lake	DynamoDB, Redis
Hopsworks	Managed / open source	S3, HDFS	RonDB (MySQL Cluster)
Vertex AI FS	GCP managed	BigQuery	Bigtable
SageMaker FS	AWS managed	S3 + Glue	DynamoDB

What we will demo

- Define a feature view in Feast
- Materialize batch features to the offline store
- Point-in-time join for training dataset generation
- Online feature retrieval for serving

The key insight

A feature store is not just storage — it is a **contract** between data engineering and model serving. It defines what a feature means and guarantees consistency.

5 Quiz 2

Q1 — Point-in-Time Correctness

You are building a churn prediction model. For each user, you create a training row using their subscription status as of date D. One of your features is: *"total support tickets submitted by this user"*.

A colleague queries this feature using all tickets up to today's date. What is wrong with this? How does a feature store's point-in-time join fix it?

Q2 — Batch vs. Streaming Feature

You are building a real-time fraud detection model. Two proposed features are:

(a) User's total spend in the last 30 days

(b) Number of transactions in the last 5 minutes from this device

Which feature requires streaming computation, and which can be batch? What infrastructure does each require?

5 Quiz 2 — Answers

A1 — Point-in-Time Correctness

The problem: using all tickets up to today means the training row for date D includes tickets submitted *after* D. This is future information leakage. The model learns a pattern ("this user will submit 5 more tickets before churning") that is impossible to know at serving time.

Feature store fix: a point-in-time join ensures that for each training row labelled at date D, the feature value returned is computed from data with timestamp $< D$ only. Future data is never visible.

A2 — Batch vs. Streaming Feature

(a) Total spend in the last 30 days → batch: this aggregates over a long historical window. A nightly Spark job computes it and writes to the offline/online store. Infrastructure: data warehouse + Spark + feature store.

(b) Transactions in the last 5 minutes → streaming: this window is too short for batch. Requires a Kafka event stream + Flink (or Spark Streaming) to maintain a rolling 5-minute counter per device in real time. Infrastructure: Kafka + Flink + Redis (online store).

5 What We Covered Today

§1 Data Sources and Formats

- Four data source types and their challenges
- JSON vs. CSV vs. Parquet
- Row-major vs. column-major trade-offs
- 80–90% of industry data is unstructured

§2–3 Data Models and Storage

- Relational model, normalization
- Four NoSQL types and their ML use cases
- OLTP vs. OLAP access patterns
- Three ways to pass data between services

§4 Feature Engineering

- Features matter more than algorithms (Facebook)
- Missing values: deletion vs. imputation
- Scaling: always fit on training split only
- Data leakage: split first, then scale
- Point-in-time correctness

§5 Feature Stores

- Offline store: training, historical, Parquet/S3
- Online store: serving, sub-millisecond, Redis
- Compute once, use everywhere
- Feast, Tecton, Hopsworks

Thank You!

Bad features in, bad predictions out. Garbage in, garbage out — forever.

Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design

